

Research update — walk-forward validation scaffolding (2026-08-18)

For: whoever wires these files into the repo and runs the study (the agent with repo + Robinhood MCP access). **From:** offline design session (no repo access) + ChatPro handoff. **Status:** SCAFFOLDING. Nothing here has been run against real data. None of it is fundable evidence. Do not arm any order path on the strength of anything in this drop.

0. Read this first — what this drop is and is NOT

This is a set of **orchestration scaffolds** for the blocking validation work in `OptionScope_Research_Handoff.md` (Test 1/2/3) plus one new **unvalidated** strategy candidate. It was written **without repo access**, so every place that touches real repo code is marked `TODO(verify against repo)`. These are not bugs to be surprised by later — they are the seams where you connect scaffolding to the actual codebase, and they are deliberately loud.

A clean compile is not a finished study. These files are built to *fail loud* rather than produce a plausible-looking result. The emitter forces INCONCLUSIVE while placeholders are live; the sweep flags fragility; the reconciler surfaces silent survivorship pruning. Do not defeat those guards to hit a deadline — they are the entire point of the exercise, and the last "clean-looking" strategy (scoreBullish) passed casual inspection and then died out-of-sample.

1. Files in this drop

File	Role	Runs anything?
<code>runner/walk-forward.ts</code>	Test 1 orchestration: rolling folds (fit 12 / test 6, ≥6 folds), regime labeling, survivorship reconciliation, ladder+fixed exits, verdict logic	Orchestrator — calls the real replay harness
<code>runner/param-sweep.ts</code>	Test 2: parameter stability sweep, one param at a time, walk-forward per grid point (single-window fallback self-labels as degraded)	Orchestrator
<code>runner/wider-universe.ts</code>	Test 3: 50–100 symbol run; reconciliation is the actual integrity control here	Orchestrator
<code>runner/handback-emitter.ts</code>	Writes <code>optionscope_handback.json</code> + <code>test1_folds.csv</code> in the exact Section 5	Writer only

File	Role	Runs anything?
	schema; runs an integrity gate before writing	
<code>runner/score-swing-dip.ts</code>	NEW, UNVALIDATED: watchlist "buy-low" swing scanner (price/volume/EMA). A hypothesis, not an edge	Scoring fn + scanner
<code>NOTE_TO_AGENT_walk_forward.md</code>	The full correction/gap list the running agent must work through	—
<code>check_research_slice.ps1</code>	Dry-run for the bundling script: lists what would be zipped, hunts for the bar-pull generator	Read-only inspection

The authoritative to-do list is `NOTE_TO_AGENT_walk_forward.md`. This README is the map; that note is the instructions.

2. Do this in order

1. **Run `check_research_slice.ps1` before building the bundle.** Confirm the bar-pull/cache-generator script is actually included (the packaging script hardcodes the *output* `bars-daily.json` but only grabs the *generator* if its filename matches a regex). If the checker prints a generator in red ("MISSING FROM BUNDLE"), add it to `$always` in `make_optionscope_research_slice.ps1` first. Without it the agent cannot extend bars to 2020 and the study stalls at step 1.
2. **Extend the bar cache to $\geq 2020-01-01$** via the same MCP pull method that produced the existing cache (handoff Section 2b). The scaffolds assume this is already done.
3. **Add the shared exports.** `param-sweep.ts` and `wider-universe.ts` import `buildFolds`, `labelRegime`, `reconcileUniverse`, `summarizeTrades` from `walk-forward.ts` (currently private). Export them, or lift them into a shared module all three import. **Do not duplicate** — two copies of fold logic will drift and void the cross-test comparison.
4. **Resolve every `TODO(verify against repo)`.** The load-bearing ones: `runReplay` signature (all four files assume the same shape — fix once, and confirm it uses the existing `visibleBars()` no-lookahead path, not a reimplementations); the `Bar` and `TradeResult` shapes; `scoreMeanReversion`'s real parameters; and — for `score-swing-dip.ts` — the real RSI/EMA/ATR imports (reuse the exact ones `scoreMeanReversion` uses, or the head-to-head comparison isn't on a level field).
5. **Wire in-sample expectancy.** `walk-forward.ts` deliberately leaves it null, which forces INCONCLUSIVE. Compute it from THIS run's own fit windows. Do **not** reuse the old 2-fold +\$2.61 / +\$1.23 figures from NCI_MEMORY 7c.

6. **Run Test 1 → Test 2 → Test 3**, then hand results + the real commit SHA to `emitHandback`.
 7. **(Optional) Run `scoreSwingDip` through the same Test 1/2 gauntlet** and compare head-to-head against `scoreMeanReversion` on identical folds. It earns a slot only if it beats or matches the already-validated strategy.
-

3. Hard rules carried over from the handoff (do not relax)

- **No lookahead, ever.** Use `visibleBars()`. A lingering `as any` on the `runReplay` call means the no-lookahead guarantee is unverified — remove it once typed.
 - **Read-only.** No live or paper broker orders. Do not touch `runner/gates.ts`-gated paths or `runner/config.ts` risk values. Sweep params only in the harness invocation, never by editing committed config.
 - **No survivorship bias.** For Test 3 and the swing scanner especially: a symbol that returns no bars may be a real delisting, not a data gap. Check each by hand. A watchlist/universe chosen with hindsight invalidates the result.
 - **Report negative/null results plainly.** INVALIDATED or INCONCLUSIVE is an expected, useful outcome. The emitter's gate can only make a verdict *more* conservative — keep it that way.
 - **Commit SHA is mandatory** (`meta.repo_commit_sha`). The emitter blocks a missing/invalid one. Run against a committed state, not a dirty tree.
 - `scoreBullish` is **dead**. Negative-control use only.
-

4. The thing not to lose sight of (scope boundary)

Test 1 decides whether the **equities** mean-reversion path is arguable. It says **nothing** about the options contradiction (NCI_MEMORY 7d): the proven edge is short premium, and the agentic MCP only places long single-leg options — the -\$49k / 23%-win category. **A SURVIVED Test 1 is not permission on the options side.** Keep the two conclusions separate in anything reported upward, or a good equities result gets misread as a green light on the exact trade type that lost the most money.

Likewise `score-swing-dip.ts` is a candidate for the equities path only, and only if it survives. Until then it stays out of the compounding plan — the diversified long-term index position is the compounder; a validated swing scanner would be a small gated input, not the engine.

Validation scaffolding landed (Test 1/2/3 + a new unvalidated swing candidate). Nothing run yet. Agent must extend bars to 2020, resolve the `TODO(verify)` seams, wire in-sample expectancy, then run. No order path arms until Test 1 returns SURVIVED with a stable Test 2 sweep — and even then, equities only, options contradiction unresolved.